



Facultad de Ingeniería
Escuela de Ingeniería Informática

DESARROLLO DE UN LLM PARA LA GENERACIÓN DE CÓDIGO DESDE HISTORIAS DE USUARIO Y DIAGRAMAS DE CLASE.

Por

Maximiliano Jesús Espíndola Manríquez

Trabajo realizado para optar al Título de
INGENIERO (CIVIL) EN INFORMÁTICA

Prof. Guía: René Noël

Febrero 2025

Resumen

El desarrollo de software fue fortalecido por el impacto profundo de la transformación digital en diversas industrias, siendo un área crucial en la era de mercados globalizados. Este fenómeno fue acelerado por tecnologías como el comercio electrónico, las redes sociales y los servicios de streaming. En consecuencia, la capacidad de crear productos de software de forma rápida y eficiente se convirtió en un factor crítico para la competitividad empresarial.

Sin embargo, muchas organizaciones enfrentan dificultades significativas en la optimización de sus procesos de desarrollo. La traducción manual de requisitos de negocio se relaciona con la dificultad de expresar requisitos complejos en forma clara y precisa usando lenguaje natural, y la implementación en sistemas funcionales consume tiempo y recursos, resultando en soluciones poco escalables, difíciles de mantener y costosas de desarrollar. A esto se suma la complejidad de mantener una trazabilidad a largo plazo del código desarrollado por otras personas, especialmente en proyectos donde la documentación fue escasa o inexistente. Esta falta de trazabilidad complica las actualizaciones, el mantenimiento y la adaptación del software a nuevos contextos, aumentando el riesgo de errores, costos imprevistos y dificultades de expresar requisitos complejos en forma clara y precisa utilizando lenguaje natural.

Existen diversos enfoques para mitigar el problema planteado. Por un lado, el desarrollo dirigido por modelos facilita la especificación precisa y completa de sistemas complejos; por otro, los Grandes Modelos de Lenguaje (LLMs) permiten la generación automática de código. Este trabajo combina ambos elementos a través de un agente especializado que utiliza LLMs junto con técnicas de especialización para automatizar el desarrollo de software mediante asistentes de chat. Dichos modelos transforman descripciones de software redactadas en lenguaje natural en modelos conceptuales, posibilitando la implementación automatizada de prototipos funcionales.

Índice general

Resumen	II
1. Introducción	1
2. Marco Conceptual y Estado del Arte	3
2.1. Marco Conceptual	3
2.1.1. Procesamiento del lenguaje natural (NLP)	3
2.1.2. Grandes modelos de lenguaje (LLMs)	4
2.1.3. Ajuste fino de los llm(Fine-Tuning)	5
2.1.4. Generación aumentada de recuperación (RAG)	6
2.1.5. Desarrollo impulsado por modelos(MDD)	9
2.2. Estado del arte	9
3. Definición del Problema	13
3.1. Formulación del problema	13
3.2. Solución propuesta	14
3.3. Objetivos	15
3.3.1. Objetivo general	15
3.3.2. Objetivos específicos	15
3.4. Importancia del trabajo	15
3.5. Metodología	17
3.6. Resultados esperados	18
4. Diseño de la solución	20
4.1. Solución del problema	20
4.2. Técnicas de análisis	23
4.2.1. Descripción de las técnicas de Análisis	23
4.2.2. Almacenamiento en bases de datos vectoriales	24
4.2.3. Almacenamiento de estructuración de datos en formato json	24
4.3. Análisis de datos utilizando bases de datos vectoriales	25
4.4. Técnicas de visualización de datos	26

5. Experimentación	28
5.1. Diseño de experimentos	28
5.2. Datasets y/o casos de estudio	29
5.3. Interpretación de resultados	29
5.4. Procedimientos experimentales	29
5.5. Resultados esperados	30
6. Implementación	31
6.1. Software Utilizado	31
6.2. Hardware utilizado	32
6.3. Lenguajes de programación	32
6.4. Estrategia de implementación	32
6.5. Visualizaciones	34
7. Conclusiones	38
7.1. Conclusiones	38
Bibliografía	40

Índice de figuras

2.1.	[1] Representación distribuida para la transformación semántica de análisis NLP	4
2.2.	[2] Explicación de la arquitectura de los transformadores	5
2.3.	[3] Visualización del proceso de ajuste fino	6
2.4.	[4] Visualización del enfoque de RAG	7
2.5.	[5] Flujo del modelo RAG	8
3.1.	Metodología de desarrollo.	17
3.2.	Respuesta de LLM.	19
4.1.	Modelo de la solución del problema	21
4.2.	Flujo de trabajo para la generación de artefactos de software utilizando un asistente LLM en un dominio conocido.	22
6.1.	Identificación de modelos	35
6.2.	Dataset	36
6.3.	Repositorio de microservicios aplicación backend de microservicios	37

Índice de tablas

2.1. Artículos relevantes sobre la transformación de requerimientos en modelos de dominio y código funcional.	11
2.2. Artículos relevantes sobre la transformación de requerimientos en modelos de dominio y código funcional.	12
4.1. Ejemplo de Tabla de Errores Codificados	27

Capítulo 1

Introducción

La transformación digital[6] ha impulsado a las empresas de desarrollo de software a adaptarse rápidamente a las exigencias de un mercado cada vez más competitivo y globalizado. Organizaciones líderes como Meta[7] , Amazon [8] y Spotify [9] han demostrado que su capacidad para lanzar nuevos productos de software de manera ágil y eficiente es esencial para mantenerse competitivas. Sin embargo, este entorno acelerado también ha incrementado los desafíos asociados a la creación de software, donde los factores críticos de éxito, como el tiempo de desarrollo y la escalabilidad, con las características del proyecto es esencial para garantizar el éxito de los proyectos de desarrollo de software, especialmente en entornos que combinan metodologías ágiles y tradicionales. [10] .

A pesar de los avances tecnológicos, el problema constante es la necesidad de desarrollar software como desarrollo de software con un enfoque hacia el producto [11], de manera ágil y rápida para cumplir con las demandas del mercado, mientras se garantizan soluciones escalables y adaptables. Las empresas que no logran cumplir con estos objetivos corren el riesgo de perder relevancia o volverse obsoletas. Además, se estima que alrededor del 80 % de las características de los productos de software rara vez o nunca se utilizan [12], lo que subraya la importancia de optimizar los recursos y reducir los costos innecesarios. [13]

En este contexto, los Grandes Modelos de Lenguaje (LLMs) han surgido como una herramienta prometedora para automatizar la generación de software [14]. Los LLMs son modelos de inteligencia artificial entrenados con grandes cantidades de datos textuales, que tienen la capacidad de generar código a partir de descripciones en lenguaje natural. Esto permite transformar historias de usuario en modelos de dominio de sistemas de información, que luego son implementados como código funcional desarrollado sin necesidad de intervención humana en las fases más técnicas del desarrollo [15]. Especificar requerimientos en lenguaje natural para un LLM enfrenta los mismos problemas previamente identificados, como la imprecisión y la falta de completitud, especialmente en sistemas complejos. Una alternativa prometedora es el desarrollo dirigido por modelos, que permite especializar un LLM para mejorar la especificación de requerimientos, garantizando escalabilidad

y trazabilidad en las características implementadas.

La presente investigación se centra en el desarrollo de un asistente especializado basado en Grandes Modelos de Lenguaje (LLMs), utilizando la arquitectura Retrieval-Augmented Generation (RAG) [4]. Este enfoque permite automatizar la creación de software al transformar descripciones en lenguaje natural en artefactos de software. El asistente recibe como entrada historias de usuario redactadas en lenguaje natural, las cuales serán procesadas para generar modelos de dominio UML, y posteriormente código funcional. La arquitectura RAG combina técnicas de recuperación de información y generación contextual, donde las historias de usuario son convertidas en vectores semánticos para recuperar conocimiento relevante de una base de datos previamente indexada. Esta información se usa como contexto para que el asistente produzca modelos conceptuales y código alineado con los requerimientos especificados. Los impactos esperados incluyen la optimización del tiempo de desarrollo, la reducción de intervención manual y la garantía de escalabilidad, adaptabilidad y trazabilidad en el software generado. Los resultados esperados incluyen la creación de prototipos funcionales completamente automatizados, lo que reducirá significativamente los tiempos de entrega y los costos de desarrollo. A largo plazo, se espera que este enfoque contribuya a transformar la manera en que las empresas abordan la creación de software, mejorando su capacidad para adaptarse a las demandas dinámicas del mercado y brindando una ventaja competitiva considerable.

Desde un punto de vista científico, este proyecto busca no solo validar la capacidad de los LLMs en el campo del desarrollo de software, sino también explorar nuevas formas de integrar estas tecnologías en diferentes contextos empresariales.

La estructura de este trabajo se organiza de la siguiente manera: en el Capítulo 1 se presenta la introducción, donde se contextualiza el problema y se destacan los objetivos de la investigación. El Capítulo 2 aborda el marco conceptual y el estado del arte, describiendo conceptos clave como los Grandes Modelos de Lenguaje (LLMs), la arquitectura Retrieval-Augmented Generation (RAG) y el desarrollo dirigido por modelos (MDD), que sustentan la solución propuesta. En el Capítulo 3 se define el problema, se describe la solución propuesta y se establecen los objetivos específicos, así como la importancia del trabajo. El Capítulo 4 detalla el diseño de la solución, explicando el flujo de trabajo del asistente basado en LLMs y las técnicas empleadas para la generación de artefactos de software. En el Capítulo 5, se describe el diseño experimental, los casos de estudio utilizados y la metodología para evaluar la efectividad de la solución propuesta.

Capítulo 2

Marco Conceptual y Estado del Arte

En este capítulo se aborda el tema central de la investigación, presentando los conceptos clave y la teoría que respaldan el uso de Grandes Modelos de Lenguaje (LLMs) en el desarrollo automatizado de software. Este enfoque se fundamenta en el creciente impacto del procesamiento del lenguaje natural (NLP, por sus siglas en inglés) como una herramienta esencial para interpretar y transformar descripciones en lenguaje natural en modelos y prototipos funcionales. Además, se analizan las técnicas de ajuste fino (fine-tuning), la arquitectura Retrieval-Augmented Generation (RAG) y el desarrollo impulsado por modelos (MDD, Model-Driven Development), que son pilares técnicos de la solución propuesta. Un elemento clave adicional es la ingeniería de prompts, que permite aprovechar al máximo las capacidades de los LLMs para generar resultados precisos y relevantes. El estado del arte también incluye un análisis comparativo de estudios y trabajos recientes que exploran cómo los LLMs pueden transformar requisitos en lenguaje natural en artefactos funcionales. Se discuten enfoques actuales, como el uso de RAG en sistemas de preguntas y respuestas, el ajuste fino para dominios específicos y la integración de MDD en la automatización del desarrollo de software. Finalmente, se destaca la ingeniería de prompts, una técnica clave que permite optimizar la interacción con los modelos para obtener resultados precisos, asegurando que las salidas generadas por los LLMs cumplan con los requisitos específicos del sistema.

2.1. Marco Conceptual

2.1.1. Procesamiento del lenguaje natural (NLP)

Es un subcampo de la inteligencia artificial que se centra en la interacción entre los ordenadores y el lenguaje humano. El objetivo principal del NLP es permitir que las máquinas procesen y entiendan datos textuales o hablados de la misma manera que lo hace un ser humano. Para ello, NLP combina lingüística computacional [16], machine learning,

y deep learning [17] para interpretar, analizar, y generar lenguaje natural. Esto lo hace utilizando "transformers" que son un tipo de arquitectura de redes neuronales introducida en 2017 [18] para el procesamiento de lenguaje natural (NLP). Su diseño se basa en el mecanismo de atención que permite que el modelo asigne diferentes pesos a diferentes palabras en una secuencia, enfocándose en las más relevantes para comprender el contexto, sin importar su posición en la oración. A diferencia de las redes recurrentes, los transformers pueden procesar secuencias de texto de manera paralela, lo que mejora significativamente su eficiencia. Son la base de modelos avanzados como GPT-3 [19], BERT [20], y T5 [21].

En el contexto del desarrollo de software, el NLP juega un papel crucial al permitir que los desarrolladores describan funcionalidades o características del software en lenguaje natural, y que estas descripciones puedan ser transformadas automáticamente en código funcional o documentación técnica. Herramientas como GitHub Copilot [22] o OpenAI Codex[23] son ejemplos de cómo el NLP, a través de los Grandes Modelos de Lenguaje (LLMs), está facilitando el desarrollo de software mediante la generación automática de código a partir de descripciones textuales.

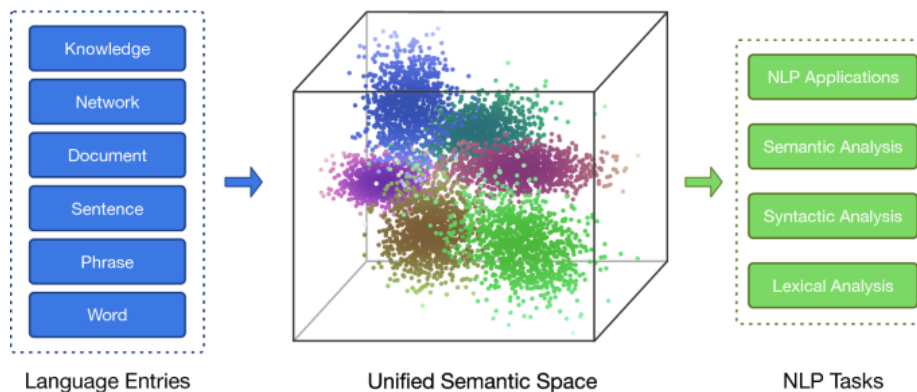


Figura 2.1: [1] Representación distribuida para la transformación semántica de análisis NLP

2.1.2. Grandes modelos de lenguaje (LLMs)

El NLP es fundamental para los LLMs (Large Language Models), que se entrenan con grandes volúmenes de datos textuales y pueden realizar tareas como generación de texto, traducción automática, o clasificación de texto. Esto se hace mediante una combinación de técnicas de incrustación. Las incrustaciones son las representaciones de tokens, como frases, párrafos o documentos, en un espacio vectorial de alta dimensión, donde cada dimensión corresponde a una característica o atributo aprendido de la lengua.

El proceso de incrustación tiene lugar en el codificador. Debido al enorme tamaño de los LLM, la creación de incrustaciones requiere una amplia formación y recursos considerables. Sin embargo, lo que diferencia a los transformadores de las redes neuronales

anteriores es que el proceso de incrustación es altamente paralelizable, lo que permite un procesamiento más eficaz. Esto es posible gracias al mecanismo de atención [18].

El mecanismo de atención permite a los transformadores predecir palabras bidireccionalmente [20], es decir, basándose tanto en las palabras anteriores como en las siguientes. El objetivo de la capa de atención, que se incorpora tanto en el codificador como en el decodificador, es captar las relaciones contextuales existentes entre las distintas palabras de la frase de entrada. [2]

En el desarrollo de software, esta capacidad permite automatizar la creación de código o incluso la generación de pruebas unitarias basadas en descripciones en lenguaje natural. En la figura 2.2 se muestran que estos modelos están diseñados para comprender, generar, y manipular lenguaje natural de manera efectiva, basándose en patrones aprendidos de datos masivos. Su tamaño se refiere al número de parámetros (variables ajustables) que contiene el modelo, lo que les permite manejar tareas complejas de lenguaje.

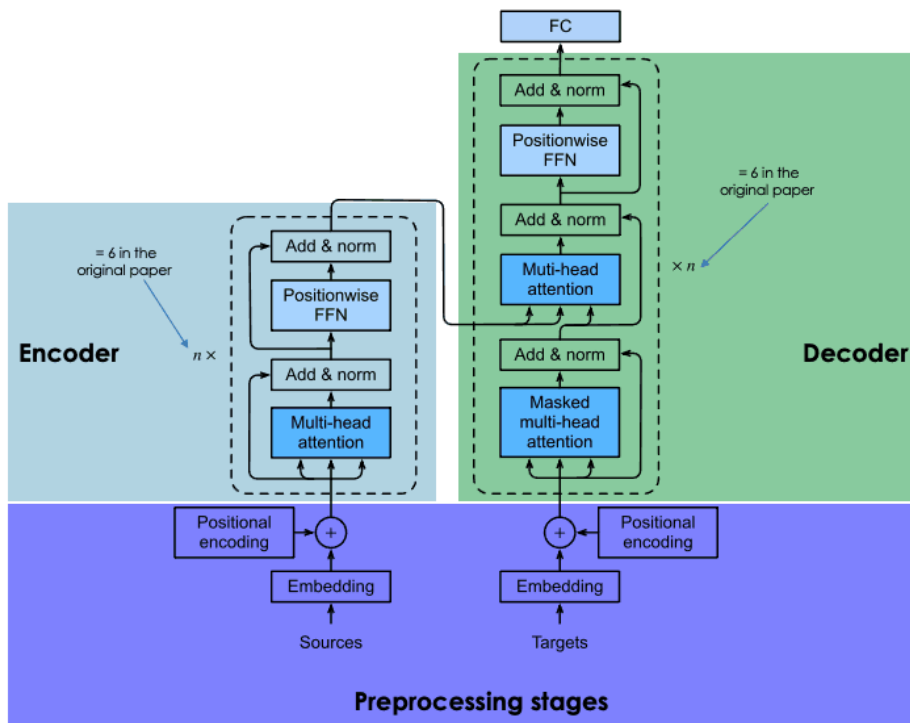


Figura 2.2: [2] Explicación de la arquitectura de los transformadores

2.1.3. Ajuste fino de los llm(Fine-Tuning)

El ajuste fino o fine-tuning de los Modelos de Lenguaje Grande (LLMs) es el proceso de adaptar un modelo preentrenado para realizar tareas específicas mediante un entrenamiento adicional con un conjunto de datos especializados. Este proceso que se puede

visualizar en la Figura 2.3 mejora el rendimiento del modelo en dominios concretos, reduciendo el tiempo y los recursos necesarios en comparación con el entrenamiento desde cero. Fine-tuning permite que los LLMs se adapten a tareas como análisis de sentimientos o respuestas a preguntas, haciéndolos más efectivos en aplicaciones especializadas. [3] Este proceso es crucial para especializar un modelo asistente para una tarea en específica porque previamente entrenado con datos generales, se puede adaptar a tareas o dominios específicos. Sin el ajuste fino, el modelo podría no responder correctamente a tareas especializadas o manejar con precisión el contexto particular de ciertos dominios. Fine-tuning optimiza el rendimiento del asistente, mejorando su capacidad para generar respuestas más relevantes y precisas en aplicaciones como atención al cliente, consultas técnicas o asistencia médica, donde el conocimiento específico es esencial. [24]

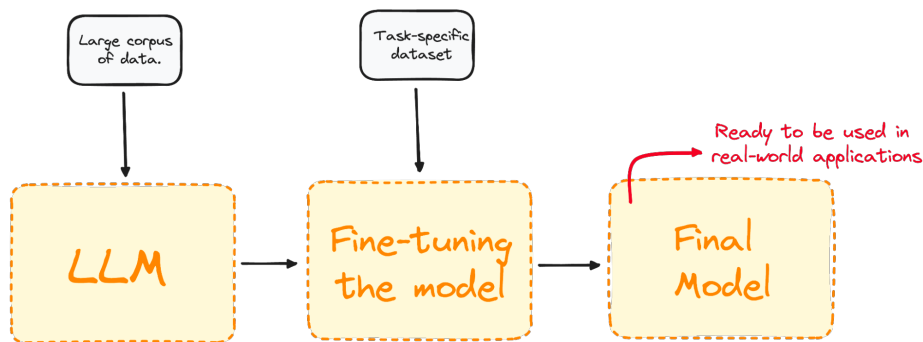


Figura 2.3: [3] Visualización del proceso de ajuste fino

2.1.4. Generación aumentada de recuperación (RAG)

La arquitectura Retrieval-augmented generation (RAG) [25] combina dos componentes principales: *recuperación de información relevante* (retrieval) y *generación de texto* (generation). Este enfoque mejora la capacidad de los modelos de lenguaje para producir resultados precisos y contextualizados, especialmente en tareas complejas como la generación de documentación técnica o la interpretación de requerimientos.

1. Fase de recuperación (Retrieval)

- La entrada del usuario se convierte en un vector utilizando un modelo de incrustaciones (*embeddings* BERT [20]).
- Se realiza una búsqueda semántica en una base de conocimiento indexada previamente, recuperando los documentos más relevantes (top-k).

2. Fase de generación (Generation)

- Los documentos recuperados se concatenan con la entrada del usuario.
- El modelo generador de texto (e.g., GPT [26]) utiliza esta información contextualizada para producir una respuesta coherente y relevante.

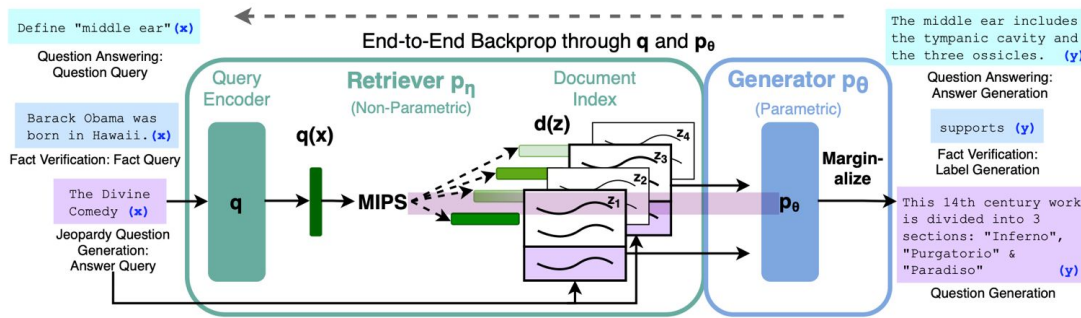


Figura 2.4: [4] Visualización del enfoque de RAG

La figura muestra 2.4 cómo se combina un recuperador preentrenado (encoder de consultas + índice de documentos) con un modelo preentrenado (generador), ajustado de manera integral. Para una consulta x , se utiliza *Maximum Inner Product Search (MIPS)* para encontrar los K documentos más relevantes (z_i). La predicción final y se obtiene tratando z como una variable latente y marginalizando las predicciones del modelo sobre los documentos recuperados.

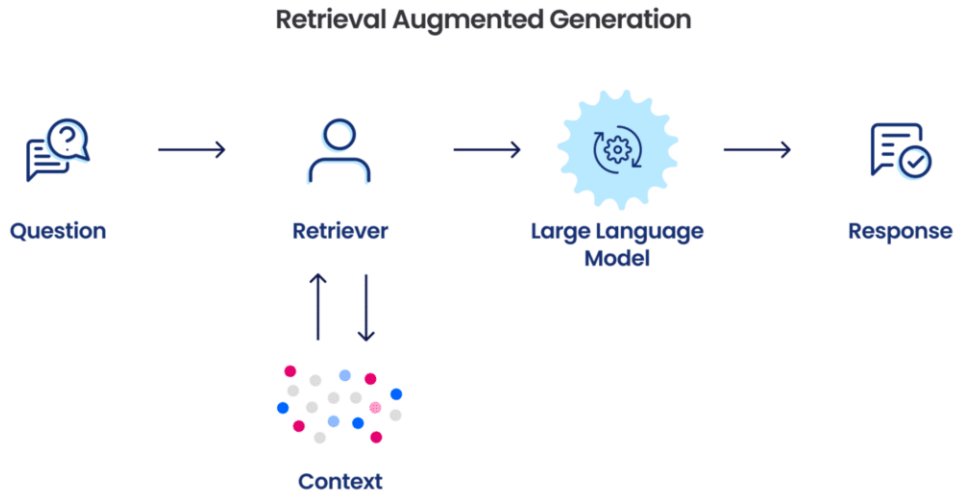


Figura 2.5: [5] Flujo del modelo RAG

En la figura 2.5 se muestra que el flujo de rag es particularmente útil en tareas de generación de respuestas donde es fundamental acceder a grandes cantidades de datos externos, como en sistemas de preguntas y respuestas o asistencia basada en documentación técnica.

El diagrama muestra la combinación de recuperación de información y generación de respuestas mediante un modelo de lenguaje. Los pasos son los siguientes:

1. **Query al modelo de incrustaciones (*embeddings*):** Se pasa una consulta (*query*) al modelo de incrustaciones para representar su significado en forma de un vector de consulta embebido.
2. **Transferencia a la base de datos vectorial o índice disperso:** El vector de consulta generado se transfiere a una base de datos vectorial (Vector DB) o un índice disperso, como BM25.
3. **Recuperación de fragmentos relevantes:** Usando un algoritmo de recuperación, se obtienen los k fragmentos (*chunks*) más relevantes de la base de datos.
4. **Envío al LLM:** El texto de la consulta y los fragmentos recuperados se envían a un *Large Language Model* (LLM) para su procesamiento.
5. **Generación de la respuesta:** El LLM utiliza el contenido recuperado como contexto para generar una respuesta basada en el *prompt* proporcionado.

2.1.5. Desarrollo impulsado por modelos(MDD)

Model Driven Development (MDD) es una metodología de desarrollo de software que pone un fuerte énfasis en la creación de modelos como la principal forma de representar y manipular los sistemas de software. En lugar de escribir código manualmente desde el principio, en MDD los modelos abstractos son la fuente principal de información para el desarrollo, y a partir de ellos se genera código automáticamente o semi-automáticamente.

El objetivo de MDD es mejorar la eficiencia, calidad y mantención del software mediante el uso de modelos que pueden ser fácilmente entendidos por los desarrolladores y otras partes interesadas (stakeholders), y luego transformados en código funcional utilizando herramientas automáticas. Esto permite un enfoque más estructurado y flexible en el desarrollo de software. Los modelos son tratados como los artefactos clave del desarrollo, y todas las decisiones importantes se toman basándose en ellos. Los modelos son transformados automáticamente en código mediante herramientas que interpretan estos modelos y generan el software final. Permite a los desarrolladores trabajar en un nivel más alto de abstracción, enfocándose en la lógica del negocio y los requisitos, dejando los detalles de implementación para las herramientas automatizadas. Dado que el modelo sigue siendo el punto central, cualquier cambio o actualización se hace a nivel del modelo, y el código se actualiza automáticamente para reflejar estos cambios [27].

En el contexto del desarrollo de software automatizado, como se describe en el proyecto de un asistente LLM, MDD juega un papel crucial. Los modelos conceptuales generados por el asistente LLM sirven como la fuente de entrada para la automatización, lo que permite transformar estos modelos en código sin la intervención manual en muchas de las fases técnicas del desarrollo.

2.2. Estado del arte

En esta sección se recopilan y analizan diferentes trabajos relacionados con el uso de Grandes Modelos de Lenguaje (LLMs) en el desarrollo automatizado de software. Se han buscado estudios enfocados en la completación de código, explorando cómo los LLMs permiten la transformación de requerimientos en código funcional. Además, se examina el funcionamiento de la arquitectura Retrieval-Augmented Generation (RAG), la seguridad y privacidad de los LLMs, y sus aplicaciones en ingeniería de software, especialmente integrando modelos dirigidos por software (MDD). También se consideran investigaciones que abordan los desafíos y oportunidades en la era del Big Data, los avances tecnológicos aplicados al NLP y la ingeniería de requerimientos, y cómo la inteligencia artificial se integra en el desarrollo dirigido por modelos. Finalmente, se estudian enfoques que generan automáticamente modelos desde requerimientos en lenguaje natural, crear trazabilidad en

el MDD, explorar perspectivas industriales de MDD y proponer soluciones para pasar historias de usuario a código.

La búsqueda se realizó principalmente en Google Scholar, ResearchGate, Elsevier, ScienceDirect y arxiv.org utilizando palabras clave como *MDD*, *user stories to code*, *LLMs applying to software development*, además de revisar artículos y fuentes proporcionadas por el profesor como referencias iniciales.

Al analizar los trabajos existentes, se observa que existen estudios similares que abordan la generación automática de código utilizando LLMs y el desarrollo dirigido por modelos. Sin embargo, estos trabajos suelen enfocarse en aspectos específicos, como la transformación de requerimientos en modelos de dominio o la completación de código, sin integrar completamente un flujo automatizado y completo que pase de descripciones en lenguaje natural a código funcional. Asimismo, se identificaron otros trabajos que, aunque no son similares en forma, abordan el mismo problema desde perspectivas parciales, como el uso de arquitecturas RAG o técnicas de NLP aplicadas a la ingeniería de requerimientos. La principal limitación de los estudios actuales radica en que no consolidan un enfoque integrado y especializado que permita la generación escalable, trazable y eficiente de software a partir de descripciones en lenguaje natural, aprovechando los LLMs en un entorno automatizado. No obstante, los trabajos existentes ofrecen bases importantes que se pueden aprovechar en el presente proyecto, como las técnicas de ajuste fino (fine-tuning), el uso de arquitecturas RAG para enriquecer el contexto de generación y las metodologías de MDD para estructurar la transformación de modelos en código. Estos aportes permiten sentar las bases para un asistente especializado que automatice todo el proceso, garantizando un desarrollo de software eficiente y alineado con las necesidades actuales.

Título	Autores	Año	Descripción
Large Language Models for Code Completion: A Systematic Literature Review [28]	Rasha Ahmad Husein a, Hala Aburajouh a, Cagatay Catal	2024	Revisión sistemática sobre el uso de LLMs en la autocompletación de código.
Requirements are All You Need: From Requirements to Code with LLMs [15]	Bingyang Wei	2024	LLM especializado que, mediante el método Progressive Prompting, automatiza la generación de modelos, pruebas y código a partir de requerimientos textuales.
ARKS: Active Retrieval in Knowledge Soup for Code Generation [29]	Hongjin Su, Shuyang Jiang, Yuhang Lai, Haoyuan Wu, Boao Shi, Che Liu, Qian Liu, Tao Yu	2024	Estrategia avanzada de RAG que mejora la generación de código en LLMs mediante una recuperación activa que refina iterativamente consultas, integrando búsqueda web, documentación y retroalimentación. Mejorando la precisión en problemas de codificación.
Can ChatGPT replace StackOverflow? A Study on Robustness and Reliability of Large Language Model Code Generation [30]	Li Zhong and Zilong Wang	2024	Importancia de implementar mecanismos de validación y corrección en la generación de código, asegurando que el software resultante sea funcional y robusto, minimizando errores en la implementación de APIs.
From Natural Language to Web Applications: Using Large Language Models for Model-Driven Software Engineering [?]	Netz, Lukas; Michael, Judith; Rumpe, Bernhard	2024	LLM
Test-Driven Development for Code Generation [?]	Noble Saji Mathews; Meiyappan Nagappan	2024	LLM
Model-Driven Development in the Era of Big Data: Challenges and Opportunities	S. Dustdar, T. Hummer	2021	Analiza cómo el desarrollo dirigido por modelos (MDD) puede adaptarse a las necesidades del big data, transformando requerimientos en modelos y código escalables.

Tabla 2.1: Artículos relevantes sobre la transformación de requerimientos en modelos de

Título	Autores	Año	Descripción
Advancements in Natural Language Processing for Requirements Engineering	B. Nuseibeh, S. Easterbrook	2022	Revisión de los avances en NLP aplicados a la ingeniería de requerimientos, facilitando la transformación de descripciones en lenguaje natural en modelos de dominio.
Integrating AI Techniques in Model-Driven Development: A Systematic Review	M. Wimmer, G. Kappel	2023	Examen de cómo las técnicas de inteligencia artificial pueden integrarse en MDD para mejorar la transformación de requerimientos en modelos y código.
Leveraging Machine Learning for Automated Code Generation from Requirements	J. Whittle, J. Hutchinson	2020	Investiga el uso de técnicas de aprendizaje automático para automatizar la generación de código a partir de requerimientos de software.
Bridging the Gap between Requirements Engineering and Model-Driven Development	A. Egyed, P. Grünbacher	2019	Propuesta para integrar la ingeniería de requerimientos con MDD, facilitando la transformación de requerimientos en modelos y código.
Automatic Generation of Domain Models from Natural Language Requirements	M. Harman, Y. Jia	2018	Explora técnicas de procesamiento de lenguaje natural para convertir automáticamente requerimientos en modelos de dominio.
Enhancing Traceability in Model-Driven Development	L. Briand, Y. Labiche	2017	Aborda la importancia de la trazabilidad en MDD y propone métodos para mantener la coherencia entre requerimientos, modelos y código.
Challenges in Model-Driven Development: An Industrial Perspective	H. Störrle, J. Teuber	2016	Identifica y analiza los desafíos que enfrentan las industrias al implementar MDD para transformar requerimientos en modelos y código.
From User Stories to Code: An Agile Approach Using Model-Driven Development	P. Mohagheghi, V. Dehlen	2015	Propone una integración de metodologías ágiles con MDD para convertir historias de usuario en código funcional de manera eficiente.

Tabla 2.2: Artículos relevantes sobre la transformación de requerimientos en modelos de

Capítulo 3

Definición del Problema

3.1. Formulación del problema

El desarrollo de software enfrenta desafíos significativos debido a la falta de automatización en la transformación de historias de usuario en modelos conceptuales y, posteriormente, en código funcional. Este proceso, que usualmente depende de tareas manuales, no solo incrementa la complejidad y los tiempos de desarrollo, sino que también introduce riesgos de errores, inconsistencias y documentación obsoleta. Estas limitaciones afectan la trazabilidad desde los requisitos iniciales hasta el producto final, complicando la evolución y el mantenimiento del software[31].

El enfoque tradicional de desarrollo, caracterizado por su rigidez y dependencia de múltiples lenguajes de programación, amplifica los desafíos al exigir una inversión considerable de tiempo y recursos. Incluso con metodologías ágiles, la colaboración constante y la planificación iterativa requieren un esfuerzo organizacional significativo, lo que puede limitar la adaptabilidad a las demandas cambiantes del mercado[32].

Por lo tanto, este proyecto propone desarrollar un asistente basado en Grandes Modelos de Lenguaje (LLMs), diseñado para transformar descripciones en lenguaje natural, como historias de usuario, en modelos conceptuales que puedan ser convertidos automáticamente en código funcional, estos modelos hacen las especificaciones de sistemas complejos mas precisas. Este asistente busca no solo optimizar el proceso de desarrollo al reducir tiempos y costos, sino también asegurar trazabilidad, escalabilidad y adaptabilidad, abordando de manera efectiva los retos de la industria del software.

3.2. Solución propuesta

La propuesta de este proyecto se centra en desarrollar un asistente especializado, basado en un Modelo de Lenguaje Grande (LLM), para convertir historias de usuario redactadas en lenguaje natural en modelos conceptuales de sistemas de información (UML). Estos modelos serán transformados automáticamente en código funcional. Este enfoque está diseñado para permitir a los usuarios, incluso aquellos sin conocimientos técnicos avanzados, generar prototipos de software funcionales de manera rápida y eficiente.

Para implementar esta solución, se utilizará un enfoque basado en Retrieval-Augmented Generation (RAG), que combina las capacidades del LLM con un sistema de recuperación de información relevante para enriquecer las respuestas del modelo con datos precisos y contextualizados. Específicamente, el asistente se construirá sobre un LLM integrado mediante el framework LangChain, lo que permitirá orquestar de manera eficiente los pasos necesarios para procesar las historias de usuario, generar modelos UML y, posteriormente, traducirlos en código funcional.

El asistente sigue un enfoque de Desarrollo Dirigido por Modelos (MDD), donde los modelos abstractos, se convierten en la fuente principal para la generación automática de código. Esto no solo optimiza el ciclo de vida del desarrollo del software, sino que también permite a los desarrolladores enfocarse en tareas de diseño y arquitectura en lugar de la programación manual, mejorando significativamente la productividad.

1. Generación de Código a partir de Modelos Conceptuales

El asistente interpretará modelos conceptuales, como diagramas UML, especificaciones formales o documentos de requisitos, generando el código correspondiente en lenguajes seleccionados. Este enfoque asegura que el código sea tanto sintáctico como semánticamente funcional.

2. Proceso de Desarrollo Iterativo

El desarrollo será iterativo, con ciclos continuos de entrada de modelos conceptuales, generación de código y refinamiento. Cada iteración buscará mejorar la precisión del código generado y adaptarse a nuevas necesidades del sistema.

3. Creación de un Sistema de Prueba

Se desarrollará un sistema funcional limitado como prueba de concepto, con el objetivo de demostrar la capacidad del asistente para producir código funcional a partir de modelos conceptuales. Este sistema ayudará a identificar limitaciones y desafíos, contribuyendo al refinamiento de la metodología.

4. Integración de Fine-Tuning

El fine-tuning permitirá especializar el modelo para dominios específicos del desarrollo de software. Esto asegura precisión y adaptabilidad a contextos particulares.

5. Uso de la Arquitectura RAG

La integración de Retrieval-Augmented Generation (RAG) complementará las capacidades del asistente al permitirle acceder a información externa en tiempo real. Esto mejorará el contexto y la trazabilidad del software generado.

3.3. Objetivos

Además, los objetivos específicos deben ser consistentes con el objetivo general, y asegurar el alcanzarlo.

3.3.1. Objetivo general

Desarrollar un asistente especializado basado en un Modelo de Lenguaje Grande (LLM) que permita la conversión de historias de usuario redactadas en lenguaje natural en modelos conceptuales de sistemas de información, como diagramas UML, y su posterior traducción a código funcional. Este asistente buscará optimizar los tiempos de desarrollo, mejorar la trazabilidad entre requerimientos y código generado, y también reducir la dependencia de conocimientos técnicos avanzados.

3.3.2. Objetivos específicos

1. Caracterizar y evaluar los distintos LLM existentes, usando sus prácticas actuales en el desarrollo de software y su capacidad para clases de sistema desde historias de usuarios.
2. Diseñar un método automatizado para transformar historias de usuarios en modelo de dominio del sistema y luego a código funcional.
3. Implementar el asistente en un entorno real, validando su capacidad para generar código preciso a partir de modelos conceptuales de sistemas de información, integrando dependencias entre componentes.
4. Validar la propuesta mediante estudios de caso con aplicaciones reales de distinta complejidad, midiendo su efectividad en casos de productividad, precisión del código y reducción de la intervención manual en el prototipo funcional.

3.4. Importancia del trabajo

Este proyecto posee una gran relevancia, tanto desde una perspectiva científica como social, con un impacto directo en la eficiencia del desarrollo de software. En el ámbito

científico, la propuesta contribuye al avance de los Grandes Modelos de Lenguaje (LLMs), integrando técnicas avanzadas de fine-tuning para especializar estos modelos en la automatización del desarrollo de software mediante el enfoque de Model Driven Development (MDD). Este trabajo amplía el uso de la inteligencia artificial en la ingeniería de software, demostrando cómo los LLMs pueden transformar descripciones en lenguaje natural en código funcional y escalable, algo que hasta ahora ha sido un desafío en proyectos complejos. Además, fomenta el desarrollo de modelos personalizables que pueden adaptarse a contextos específicos, lo que abre nuevas oportunidades para su aplicación en diversos dominios del desarrollo de software.

Desde un punto de vista social y práctico, la implementación de este asistente LLM ofrece una solución que reduce significativamente los costos y tiempos de desarrollo, lo que beneficia tanto a pequeñas empresas como a grandes corporaciones. Al automatizar procesos que históricamente han requerido habilidades técnicas avanzadas, esta herramienta permite que startups, empresas emergentes y profesionales no técnicos puedan transformar sus ideas en prototipos funcionales con mayor rapidez y menor esfuerzo. Esto, a su vez, mejora la competitividad en el mercado global, democratizando el acceso a tecnologías avanzadas de desarrollo y fomentando la innovación. La posibilidad de que actores menos especializados puedan acceder a un desarrollo de software eficiente contribuye al crecimiento económico, promueve la adopción de soluciones tecnológicas y fortalece la inclusión en el uso de herramientas de alta tecnología.

3.5. Metodología

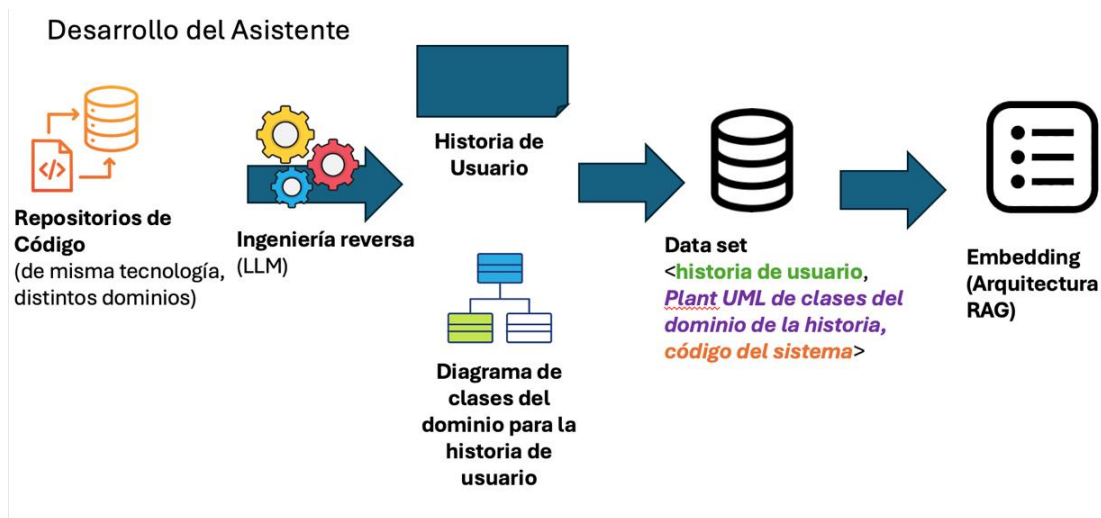


Figura 3.1: Metodología de desarrollo.

La metodología propuesta para el desarrollo del asistente basado en LLM se compone de los siguientes pasos:

1. Análisis de repositorios de código.

- Se seleccionan repositorios de código existentes que utilizan la misma tecnología pero abarcan diferentes dominios.
- Estos repositorios sirven como base para extraer patrones recurrentes de diseño y estructura de código.

2. Ingeniería reversa asistida por LLM.

- Un modelo de lenguaje grande (LLM) analiza los repositorios de código y realiza ingeniería reversa para:
 - Identificar patrones arquitectónicos comunes.
 - Extraer diagramas de clases y relaciones entre entidades del código fuente.
- Los resultados incluyen modelos conceptuales que representan el dominio del problema.

3. Procesamiento de historias de usuario.

- Se proporcionan historias de usuario como entrada al sistema. Estas historias están escritas en lenguaje natural y describen funcionalidades específicas.
- Ejemplo: “Un cliente debe poder rastrear el estado de su pedido en tiempo real.”

4. Generación de diagramas UML.

- A partir de la historia de usuario, el LLM genera modelos UML como:
 - **Diagrama de clases:** Representa entidades clave (por ejemplo, Pedido, Usuario, Estado) y sus relaciones.
 - **Diagramas de casos de uso:** Muestra las interacciones entre los actores y el sistema.
- Estos diagramas reflejan cómo los componentes implementan la funcionalidad descrita.

5. Creación del dataset.

- Se construye un dataset estructurado que incluye:
 - La historia de usuario.
 - El diagrama de clases del dominio en formato como PlantUML.
 - El código fuente asociado al dominio y la historia de usuario.
- Este dataset sirve como entrada para entrenar y mejorar el modelo.

6. Generación de incrustaciones (*embeddings*) (Arquitectura RAG).

- Se aplica una arquitectura de **Retrieval-Augmented Generation (RAG)** para:
 - Transformar la información del dataset en incrustaciones (*embeddings*) vectoriales.
 - Permitir la búsqueda y generación de artefactos específicos basados en nuevas historias de usuario.

3.6. Resultados esperados

Esta metodología busca acelerar el desarrollo de software automatizando tareas críticas como la generación de diagramas UML y código funcional, mientras asegura la trazabilidad desde los requisitos hasta los artefactos implementados.

Esta metodología estructurada permite desarrollar un sistema funcional en un tiempo limitado, aprovechando la capacidad de los LLM para acelerar el diseño y generación de artefactos de software. El problema identificado es la dificultad de transformar historias de

usuario en modelos conceptuales y código funcional, lo cual genera retrasos e inconsistencias. La metodología aborda esta problemática mediante el uso de un LLM para generar modelos y código directamente desde los requisitos. Al validar los modelos y código funcional se minimizan errores y se asegura que los entregables estén alineados con los objetivos.

Respuesta del LLM

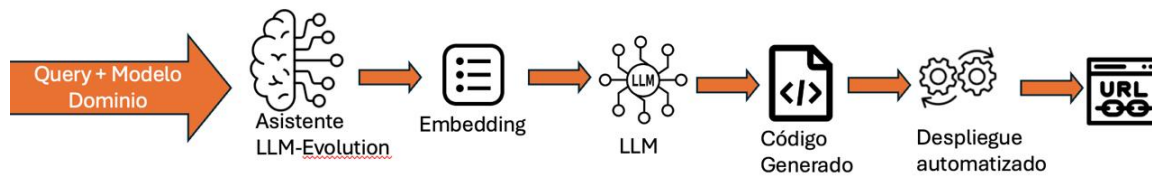


Figura 3.2: Respuesta de LLM.

Capítulo 4

Diseño de la solución

4.1. Solución del problema

La solución propuesta se basa en la metodología de Desarrollo Dirigido por Modelos (MDD), complementada con un asistente de lenguaje natural (LLM-Evolution) para automatizar la generación de código funcional a partir de modelos conceptuales y lenguaje natural. Este enfoque se centra en transformar historias de usuario en modelos de dominio y luego en prototipos navegables, asegurando trazabilidad y consistencia en el flujo de desarrollo.

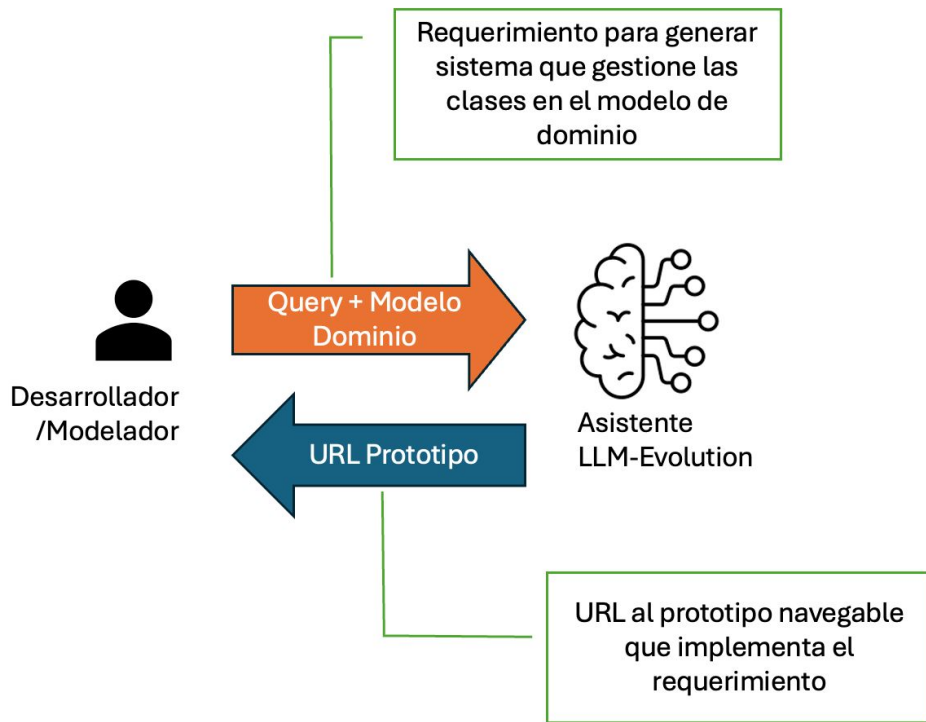


Figura 4.1: Modelo de la solución del problema

La Figura 4.1 ilustra el proceso en el cual el desarrollador solicita los requerimientos necesarios para generar un sistema que administre las clases dentro del modelo de dominio. A partir de esta solicitud, el asistente LLM procesa la información y devuelve un prototipo navegable que cumple con el requerimiento especificado. Este enfoque permite generar artefactos de software de manera eficiente, optimizando el tiempo y los recursos invertidos en el desarrollo.

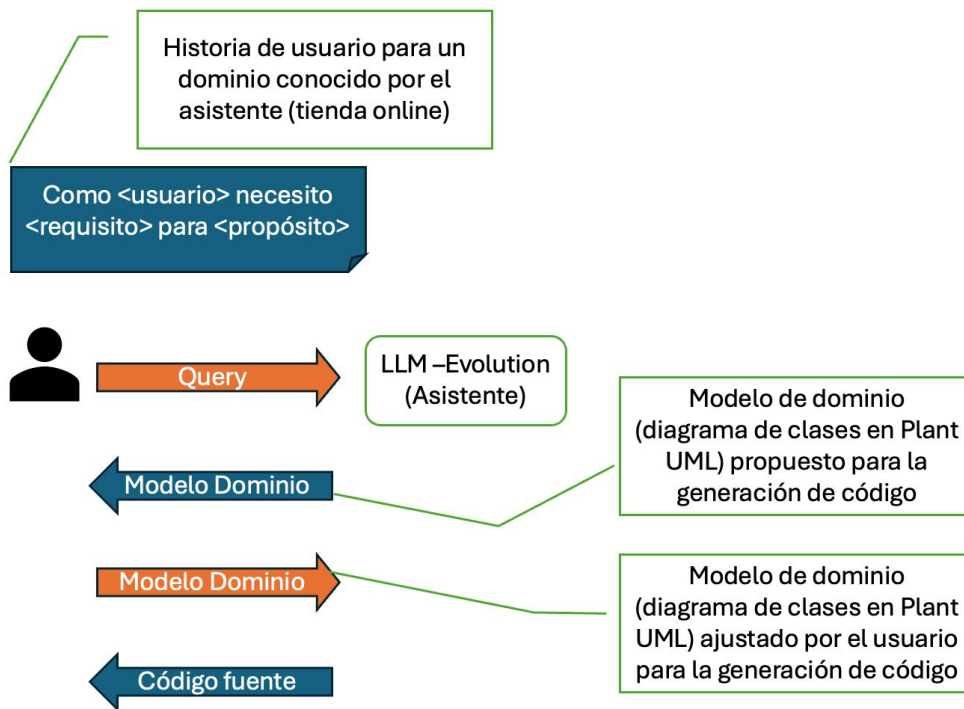


Figura 4.2: Flujo de trabajo para la generación de artefactos de software utilizando un asistente LLM en un dominio conocido.

La Figura 4.2 describe el flujo de trabajo para la generación de artefactos de software basado en un asistente de lenguaje de gran modelo (LLM) dentro del contexto de un dominio conocido, como una tienda en línea (e-commerce).

1. **Introducción de la historia de usuario:** El proceso comienza cuando el usuario o desarrollador ingresa una historia de usuario que detalla una necesidad específica dentro del dominio conocido por el asistente. Por ejemplo: *"Como cliente, necesito rastrear el estado de mi pedido para mantenerme informado sobre su progreso."* Esta historia de usuario es la base sobre la cual el asistente trabajará.
2. **Generación del modelo de dominio inicial:** El asistente LLM interpreta la historia de usuario y genera automáticamente un modelo de dominio preliminar, representado en un diagrama de clases utilizando PlantUML. Este modelo incluye entidades clave, como `Pedido`, `Estado`, y `Usuario`, junto con las relaciones y atributos necesarios para abordar el requerimiento descrito.
3. **Ajuste del modelo por el usuario:** El modelo de dominio generado por el asistente es revisado por el usuario o desarrollador. Durante esta etapa, el usuario puede realizar modificaciones o ajustes al modelo para asegurarse de que refleja fielmente los

requisitos y las particularidades del sistema. Este paso es crucial para personalizar el modelo según las necesidades específicas del proyecto.

4. **Preparación para la generación de código:** Una vez que el modelo ajustado es devuelto al asistente, este último lo utiliza como entrada para generar el código fuente correspondiente. Este código está diseñado para implementar las funcionalidades descritas en la historia de usuario de manera eficiente y alineada con el modelo de dominio ajustado.

Esta metodología asegura que los artefactos generados estén alineados con los requerimientos iniciales, optimizando el tiempo de desarrollo y facilitando la interacción entre el usuario y el asistente mediante un enfoque basado en modelos.

4.2. Técnicas de análisis

4.2.1. Descripción de las técnicas de Análisis

La técnica de análisis inicia con la **recolección de datos**, un paso esencial para evaluar la calidad del código generado mediante pruebas comparativas. Este proceso utiliza proyectos de software disponibles en repositorios de GitHub como fuente de datos. Estos proyectos incluyen carpetas denominadas `models`, que contienen las definiciones de las entidades clave del dominio del proyecto.

A partir de esta estructura, se seleccionan repositorios relevantes, como E-store, para extraer tres componentes fundamentales:

- **Historias de usuario:** Descripciones funcionales que especifican las necesidades y objetivos del sistema desde la perspectiva de los usuarios.
- **Modelos de dominio:** Representaciones conceptuales, como diagramas en *PlantUML*, que definen las relaciones y estructuras entre las entidades del sistema.
- **Modelos implementados en código:** Estructuras y clases utilizadas en la implementación del software que reflejan los conceptos del dominio.

La recolección de estos elementos permite establecer un conjunto de datos sólido para analizar cómo los modelos de dominio y las historias de usuario se relacionan con el código implementado. A su vez, estos datos facilitan la comparación y validación del código generado automáticamente, asegurando que cumpla con los requisitos y mantenga la consistencia con los modelos conceptuales y funcionales definidos.

Esta técnica asegura una **trazabilidad completa** entre las historias de usuario, los modelos de dominio y el código fuente, ofreciendo una base para las pruebas y evaluaciones del sistema desarrollado.

4.2.2. Almacenamiento en bases de datos vectoriales

Descripción: Las bases de datos vectoriales permiten almacenar representaciones numéricas (*embeddings*) de datos textuales, facilitando la búsqueda semántica y la recuperación de información relevante.

Ventajas:

- Búsqueda eficiente basada en similitud semántica.
- Escalabilidad para manejar grandes volúmenes de datos.

Ejemplos de Herramientas:

- **Pinecone:** Servicio gestionado para almacenamiento y búsqueda de vectores.
- **FAISS:** Biblioteca de Facebook para búsqueda eficiente de similitudes.

4.2.3. Almacenamiento de estructuración de datos en formato json

Descripción: Almacenar los datos en formato JSON facilita su manipulación y procesamiento, permitiendo una fácil conversión a *embeddings* y su posterior almacenamiento en bases de datos vectoriales.

Ventajas:

- Flexibilidad para representar estructuras de datos complejas.
- Compatibilidad con múltiples lenguajes de programación y herramientas de procesamiento de datos.

Ejemplo de Estructura JSON:

```
{
  "id": "123",
  "tipo": "historia_usuario",
  "contenido": "Como usuario, quiero rastrear el estado de mi pedido.",
  "metadatos": {
    "proyecto": "E-store",
    "fecha_creacion": "2024-11-22"
  }
}
```

4.3. Análisis de datos utilizando bases de datos vectoriales

El proceso de análisis de datos utilizando bases de datos vectoriales incluye varios pasos clave, descritos a continuación:

1. Generación de incrustaciones (*Embeddings*):

- **Descripción:** Transformar los datos textuales en representaciones numéricas (*embeddings*) mediante un modelo preentrenado (*Sentence Transformers* o *OpenAI embeddings*).
- **Importancia:** Las incrustaciones o *embeddings* capturan relaciones semánticas entre palabras, frases o documentos, facilitando la búsqueda y el análisis basado en significado.
- **Pasos:**
 - a) Preprocesar el texto (limpieza y eliminación de ruido).
 - b) Generar incrustaciones (*embeddings*) para cada fragmento de texto utilizando un modelo de lenguaje.
 - c) Almacenar las incrustaciones(*embeddings*) junto con metadatos relevantes en la base de datos vectorial.

2. Indexación de las incrustaciones (*Embeddings*):

- **Descripción:** Indexar las incrustaciones (*embeddings*) para garantizar una búsqueda eficiente en la base de datos vectorial.
- **Ejemplos de Herramientas:** FAISS (Facebook AI Similarity Search) y Pinecone.
- **Importancia:** Optimiza la velocidad y precisión en la búsqueda de datos relevantes.
- **Pasos:**
 - a) Configurar el índice adecuado (e.g., HNSW para búsquedas aproximadas).
 - b) Insertar las incrustaciones(*embeddings*) generados en el índice.

3. Búsqueda Semántica y Recuperación:

- **Descripción:** Buscar datos relevantes en función de su similitud semántica con la incrustación(*embeddings*) de la consulta.
- **Técnica de Análisis:** Utilizar métricas de distancia como *cosine similarity* o *dot product*.
- **Casos de Uso:**

- Encontrar historias de usuario similares.
- Recuperar plantillas de modelos UML relacionadas con un requisito.

4. Análisis Contextual de los Datos Recuperados:

- **Descripción:** Analizar los datos recuperados para extraer patrones o información relevante.
- **Cómo hacerlo:**
 - a) Visualizar la distribución de las incrustaciones(*embeddings*) utilizando herramientas como t-SNE o UMAP.
 - b) Realizar análisis estadístico sobre los metadatos (frecuencia de términos, categorías más comunes).

5. Feedback Iterativo:

- **Descripción:** Incorporar retroalimentación para mejorar la calidad de los datos y incrustaciones(*embeddings*).
- **Cómo Funciona:**
 - a) Los usuarios evalúan la relevancia y precisión de los resultados.
 - b) Ajustar las incrustaciones(*embeddings*) o el índice con base en el feedback recibido.
 - c) Refinar el modelo de generación de incrustaciones(*embeddings*) si los resultados no son satisfactorios.

6. Visualización y Reporte:

- **Descripción:** Crear dashboards o reportes para monitorear el rendimiento del sistema.
- **Herramientas Sugeridas:**
 - Elasticsearch o Kibana para análisis y visualización.
 - Streamlit o Dash para construir interfaces personalizadas.

4.4. Técnicas de visualización de datos

En esta investigación, la visualización de datos es crucial para interpretar los resultados obtenidos durante la aplicación de la metodología. A continuación, se detallan las técnicas utilizadas:

1. **Tablas de Errores Codificados:** Se emplean tablas para clasificar y visualizar los tipos de errores más comunes identificados durante el desarrollo, tales como errores de sintaxis y uso de variables no declaradas. Esto permite identificar rápidamente patrones comunes y áreas que requieren atención adicional en el refinamiento de los prompts y modelos.

Tabla 4.1: Ejemplo de Tabla de Errores Codificados

Tipo de Error	Descripción	Frecuencia
Error de Sintaxis	Variables no declaradas	15
Error Semántico	Confusión en tipos de datos	10
Error de Lógica	Iteración incorrecta	8

2. **Gráficos de Distribución de Errores:** Se implementan gráficos de barras y torta para mostrar la distribución de diferentes tipos de errores, lo que facilita comprender en qué áreas los modelos presentan mayor dificultad.
3. **Visualización de Datos Vectoriales mediante Boxplots:** Los boxplots permiten analizar la dispersión de las incrustaciones(*embeddings*) generados por los modelos. Este enfoque ayuda a identificar valores atípicos y la variabilidad en las representaciones vectoriales.
4. **Reducción de Dimensionalidad:** Se utiliza PCA (*Principal Component Analysis*) o t-SNE (*t-Distributed Stochastic Neighbor Embedding*) para reducir la dimensionalidad de las incrustaciones(*embeddings*) y visualizarlos en un espacio bidimensional o tridimensional.
5. **Visualización Combinada:** Se superponen gráficos de reducción de dimensionalidad con las etiquetas de los errores para correlacionar las características del vector con los tipos de error detectados.

Estas técnicas permiten realizar un análisis exhaustivo de los datos, visualizando patrones, outliers y la relación entre las incrustaciones(*embeddings*) y los errores del modelo, lo que facilita la optimización del sistema.

Capítulo 5

Experimentación

5.1. Diseño de experimentos

En esta sección se describe el diseño experimental utilizado para evaluar la eficacia del enfoque propuesto en este proyecto. El objetivo principal es analizar cómo las historias de usuario, combinadas con modelos de dominio en UML, afectan la generación de código del prototipo de solución en términos de consistencia, calidad y adecuación funcional.

- **Preguntas de investigación:**

- RQ1: ¿Cuál es el efecto de las historias de usuario sobre la consistencia del código del prototipo generado con respecto al modelo de dominio?
- RQ2: ¿Cuál es el efecto de las historias de usuario sobre la adecuación funcional del prototipo generado generados con respecto al modelo de dominio?
- RQ3: ¿Cuál es el efecto de las historias de usuario sobre la calidad del código del prototipo generado generados?

- **Hipótesis:**

- H1: Las historias de usuario más detalladas producen mayor consistencia del código con el modelo de dominio.
- H2: Las historias de usuario más completas generan mayor adecuación funcional del código.
- H3: Las historias de usuario claras y específicas generan un código de mayor calidad.

- **Variables:**

- Variables independientes: las historias de usuario utilizadas.

- Variables dependientes: la consistencia, calidad y funcionalidad del código generado.

5.2. Datasets y/o casos de estudio

- **Origen de los datos:** Las historias de usuario y los modelos de dominio fueron obtenidos de repositorios en GitHub que incluyen diagramas UML y archivos de definición en formato JSON.
- **Formato y almacenamiento:** Los datos se almacenaron en bases de datos vectoriales y archivos estructurados (JSON) para facilitar la manipulación y análisis.
- **Preprocesamiento:** Las historias de usuario fueron adaptadas al formato requerido por los LLM, asegurando compatibilidad y uniformidad en los experimentos.

5.3. Interpretación de resultados

Para interpretar los resultados se utilizarán las siguientes técnicas de visualización:

- **Tablas de errores codificados:** Se clasificarán y visualizarán los tipos de errores identificados en el código generado.
- **Gráficos de distribución:** Gráficos de barras y torta para mostrar la proporción de errores por tipo y complejidad.
- **Boxplots:** Representarán la distribución de métricas clave como calidad del código, adecuación funcional y consistencia.

5.4. Procedimientos experimentales

- Preparación de las historias de usuario y modelos de dominio.
- Configuración del entorno de pruebas, incluyendo bases de datos vectoriales y herramientas como SonarQube.
- Generación iterativa de código utilizando LLM con diferentes variantes de historias de usuario.
- Validación y análisis del código generado.

5.5. Resultados esperados

Se espera demostrar que las historias de usuario detalladas y estructuradas pueden facilitar la generación de código funcional, semánticamente coherente y que cumpla con las especificaciones del modelo de dominio, optimizando el proceso de desarrollo de software.

Capítulo 6

Implementación

6.1. Software Utilizado

- **GitHub**: Plataforma para el control de versiones y colaboración en el desarrollo de software. Permite almacenar, gestionar y compartir código fuente, facilitando el seguimiento de cambios.
- **Docker**: Para contenerizar los componentes del sistema, garantizando portabilidad y fácil despliegue.
- **Kubernetes**: Para la orquestación de contenedores, asegurando escalabilidad, balanceo de carga y alta disponibilidad.
- **PlantUML**: Herramienta para la generación dinámica de diagramas UML, basada en las historias de usuario procesadas.
- **PyTorch**: Framework de aprendizaje profundo utilizado para implementar y ajustar el modelo de lenguaje (LLM).
- **Spring Boot**: Framework para construir y desplegar los microservicios generados en Java.
- **FAISS**: Biblioteca para almacenamiento y recuperación semántica de embeddings generados a partir de los datos procesados.
- **Github**: Herramienta para integración y entrega continua (CI/CD), facilitando el despliegue automatizado.

6.2. Hardware utilizado

- **GPUs NVIDIA:** Para el entrenamiento y ajuste fino del modelo de lenguaje, aprovechando la capacidad de cómputo acelerado.
- **Servidores con CPUs escalables:** Para ejecutar microservicios y procesos de integración.
- **Almacenamiento en red (NFS o equivalente):** Para almacenar datasets, embeddings y salidas generadas, asegurando accesibilidad y persistencia.
- **Cluster Kubernetes:** Compuesto por nodos que incluyen recursos de CPU y GPU, orquestados para balancear la carga entre componentes.

6.3. Lenguajes de programación

- **Python:**
 - Utilizado para implementar el modelo LLM y realizar el preprocesamiento de datos.
- **Java:**
 - Lenguaje principal para los microservicios generados.
- **YAML:**
 - Lenguaje de configuración utilizado para definir los despliegues en Kubernetes.
- **PlantUML:**
 - Lenguaje para generar diagramas UML a partir de descripciones textuales. Alineado con la necesidad de representar modelos conceptuales.

6.4. Estrategia de implementación

Describir la estrategia de implementación (incluir un esquema de alto nivel para aclarar explicación). Describir las etapas del proceso. Describir y justificar los patrones estructurales utilizados (Ej.: Singleton, Facade, Mediator y Abstract Factory.), donde corresponda.

En esta etapa, se piloteará el desarrollo del asistente LLM para la generación de prototipos de software en el segundo semestre. El objetivo es diseñar, implementar y probar el

flujo completo del sistema utilizando un modelo base entrenado con historias de usuario, diagramas UML y código en Java. Esto incluye pruebas con diagramas generados mediante PlantUML y la evaluación de la funcionalidad del código generado, asegurando su integración en microservicios desplegados con Kubernetes. La estrategia de implementación se fundamenta en el enfoque Domain-Driven Design (DDD) y se empleará LLama 2 junto con técnicas de RAG para optimizar la generación de código.

■ Estrategia de Implementación

- Paso 1: Recopilación de Repositorios de Código
 - Recopilación de historias de usuario y modelos UML utilizando repositorios relevantes que tengan como tecnología el uso de JAVA y Spring boot, asegurando que los repositorios incluyan estructuras de carpetas que contengan "clases" o "diagramas" que permitan realizar ingeniería inversa de manera eficiente.
- Paso 2: Aplicación de Ingeniería Inversa
 - Ajuste del modelo LLM para analizar los repositorios seleccionados utilizando sus historias y diagramas como base para analizar, Extraer diagramas UML de clases y relaciones, generando una representación del dominio a partir del código fuente. Luego se realiza una validación iterativa para asegurar resultados precisos.
- Paso 3: Definición de Historias de Usuario
 - Redactar historias de usuario que reflejen características comunes en el dominio estudiado(comercio electronico en este caso).Incluir las historias en un formato estándar que sea compatible con el sistema (en este casoJSON).
- Paso 4: Generación del Dataset:
 - Combinar historias de usuario con los diagramas UML y el código del dominio en un dataset estructurado. Almacenar el dataset en un formato accesible y optimizado para la búsqueda y recuperación.
- Paso 5: Creación de Incrustaciones (*Embeddings*)
 - Transformar los elementos del dataset (historias de usuario, diagramas UML y código) en embeddings numéricos mediante un modelo de embeddings preentrenado. Luego Almacenar los embeddings y realizar búsquedas semánticas rápidas.
- Paso 6: Entrenamiento del LLM
 - Ajustar el modelo LLM utilizando el dataset creado, asegurando que pueda transformar historias de usuario en diagramas UML y código funcional. Validar el rendimiento del modelo con pruebas iterativas.

- Paso 7: Generación de Prototipos
 - Utilizar el LLM entrenado para generar diagramas UML y código funcional a partir de nuevas historias de usuario. Asegurar que el código generado sea funcional y se integre correctamente con el resto del sistema.
- Paso 8: Despliegue del Sistema
 - Contenerizar los componentes del sistema (generador de código, servidor PlantUML, etc.) con Docker. Orquestar el sistema en Kubernetes.
- Paso 9: Pruebas y Validación:
 - Evaluar el sistema mediante pruebas de funcionalidad, verificando la precisión de los diagramas UML y el código generado. Recoger retroalimentación y realizar ajustes iterativos al modelo y al flujo de trabajo.
- Paso 10: Documentación y Visualización
 - Documentar los resultados y el flujo de trabajo. Generar visualizaciones que reflejen el impacto de cada etapa en los resultados finales.

6.5. Visualizaciones

Se buscaron repositorios públicos en GitHub relacionados con sistemas de comercio electrónico que incluyeran microservicios. El criterio principal fue que cada repositorio contuviera una estructura bien definida con una carpeta `models` en cada microservicio. Esto permitió extraer y analizar las clases y entidades representativas de cada servicio.

Historias de Usuario

- Se formularon historias de usuario claras y específicas basadas en los casos de uso más comunes encontrados en los repositorios. Ejemplo: "Como usuario registrado, quiero poder verificar mi cuenta de correo electrónico para asegurarme de que la cuenta está activa."

Modelo UML:

- Se representaron entidades y relaciones clave utilizando PlantUML, como `User`, `Credential`, `VerificationToken`, y `Address`. Se utilizaron entidades abstractas (`AbstractMappedEntity`) para estructurar herencias comunes.

Criterios de Aceptación:

- Definidos para garantizar que cada funcionalidad cumpliera con los requisitos básicos. Ejemplo: "El sistema debe generar un token de verificación al registrarse y enviarlo por correo electrónico." Microservicios Analizados

Los microservicios identificados incluyen:

- favourite-Service: servicios favoritos de los usuarios.
- Order-Service: Gestión de pedidos.
- Payment-Service: Procesamiento de pagos.
- Product-Service: Gestión de productos.

El modelo se visualizó mediante diagramas UML, representando:

Relaciones entre entidades principales (User, Credential, Address). La interacción entre microservicios y las funciones que desempeñan en el sistema.

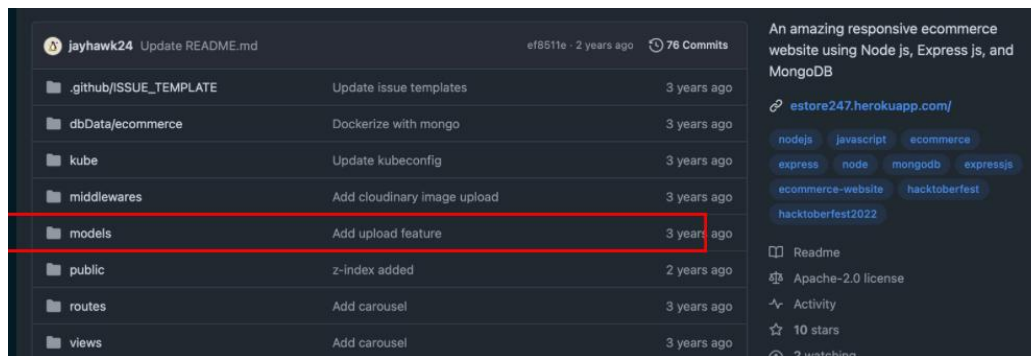


Figura 6.1: Identificación de modelos

```

{
  "id": 1,
  "service": "user-service",
  "user_history": {
    "title": "Como usuario registrado, quiero poder verificar mi cuenta",
    "description": "Cuando un usuario se registra en la plataforma, debe poder verificar su cuenta",
    "acceptance_criteria": [
      "Generación de Token de Verificación: Al registrarse, se debe generar un token de verificación",
      "Envío de Correo Electrónico: El sistema debe enviar un correo electrónico con el token de verificación",
      "Verificación del Token: Al hacer clic en el enlace de verificación, el sistema debe validar el token",
      "Manejo de Tokens Expirados: Si el token ha expirado, el sistema debe generar uno nuevo",
      "Persistencia de Datos: Los datos del usuario, credenciales y tokens deben persistir en la base de datos"
    ]
  },
  "modelo_uml": {
    "startuml": "@startuml",
    "AbstractMappedEntity": {
      "createdAt": "Instant",
      "updatedAt": "Instant"
    },
    "User": {
      "userId": "Integer",
      "firstName": "String",
      "lastName": "String",
      "imageUrl": "String",
      "email": "String",
      "phone": "String",
      "addresses": "Set<Address>",
      "credential": "Credential"
    },
    "Credential": {
      "credentialId": "Integer",
      "username": "String",
      "password": "String",
      "roleBasedAuthority": "RoleBasedAuthority",
      "isEnabled": "Boolean",
      "isAccountNonExpired": "Boolean",
      "isAccountNonLocked": "Boolean",
      "isCredentialsNonExpired": "Boolean",
      "user": "User",
      "verificationTokens": "Set<VerificationToken>"
    }
  }
}

```

Figura 6.2: Dataset

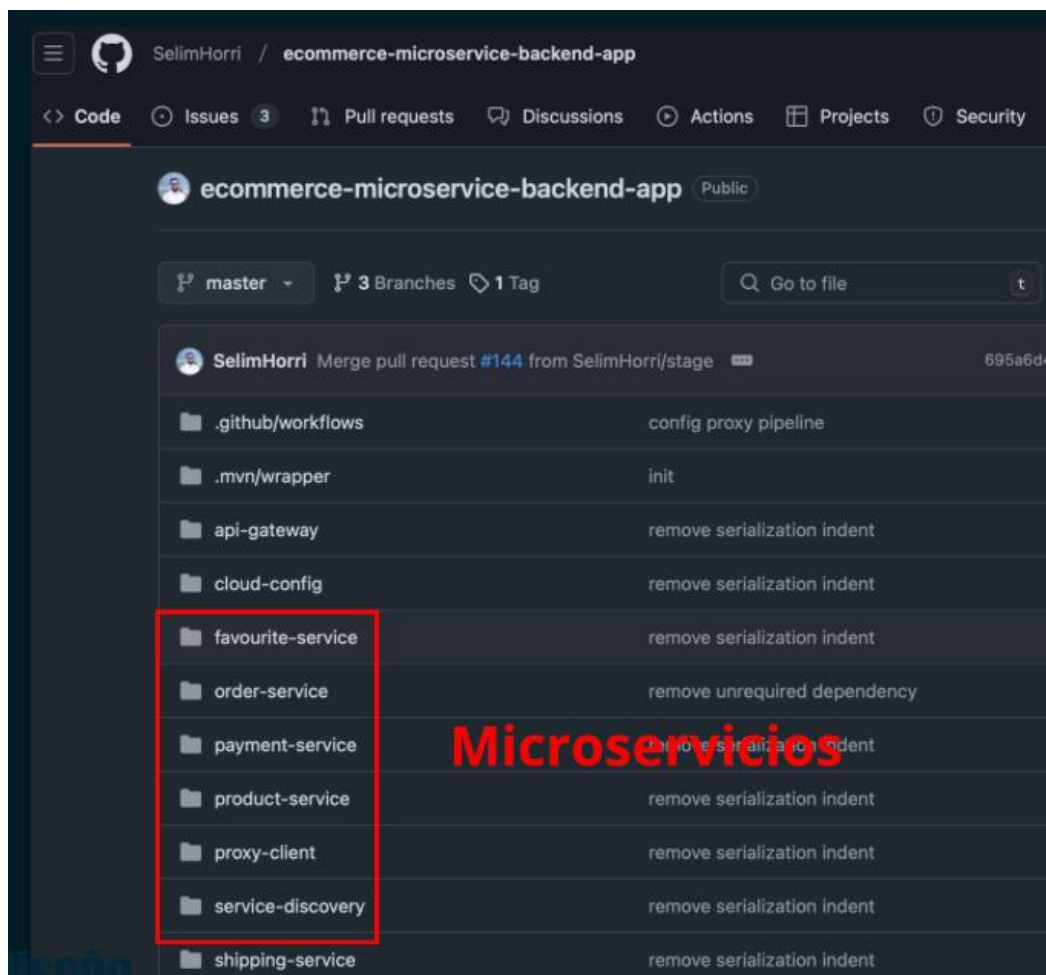


Figura 6.3: Repositorio de microservicios aplicación backend de microservicios

Capítulo 7

Conclusiones

7.1. Conclusiones

En este proyecto se desarrolló un asistente basado en LLM (Large Language Model) para transformar historias de usuario en prototipos funcionales de software. El trabajo incluyó la implementación de un modelo de aprendizaje profundo integrado con herramientas como PlantUML para diagramas y Spring Boot para microservicios, garantizando escalabilidad y precisión mediante un enfoque basado en arquitecturas modernas como RAG (Retrieval-Augmented Generation).

Descripción de cumplimiento de objetivos iniciales

- **Analizar el estado del arte de los LLMs:** Se realizó una revisión exhaustiva sobre la aplicación de modelos de lenguaje en el desarrollo de software, identificando buenas prácticas y desafíos actuales.
- **Diseñar un método para transformar historias de usuario:** Se implementó una manera que permite convertir historias en diagramas UML y código funcional, validando su efectividad en múltiples escenarios.
- **Implementar y validar el asistente en un entorno real:** No se probó el asistente en un entorno controlado, pero este objetivo será evaluado en la siguiente investigación para generando código preciso, evaluando su desempeño con estudios de caso.
- **Validar la propuesta mediante estudios de caso:** No se han realizaron pruebas en aplicaciones de distinta complejidad, se necesita demostrar mejoras en términos de productividad y precisión.

Presentación de dificultades durante el trabajo

- Los proyectos y repositorios trabajan de manera distinta las estructuras de directorios, lo que dificultó la uniformidad en la extracción de modelos.
- Implementación del dataset para la arquitectura RAG presentó desafíos, especialmente en la integración de datos heterogéneos.

Limitaciones y proyecciones del trabajo

- Durante las pruebas se observaron alucinaciones en la generación de datos y código por parte de los modelos LLM, afectando la precisión en algunos casos.
- Se recomienda optimizar la implementación del dataset para la siguiente iteración de la investigación y explorar modelos más avanzados para mejorar la capacidad de generalización del asistente.

Bibliografía

- [1] Y. Goldberg, “Representation learning and nlp,” 2020, [Online]. Available: https://www.researchgate.net/publication/342680129_Representation_Learning_and_NLP. [Online]. Available: https://www.researchgate.net/publication/342680129_Representation_Learning_and_NLP
- [2] DataCamp, “What is an llm? a guide on large language models,” 2023, [Online]. Available: <https://www.datacamp.com/es/blog/what-is-an-llm-a-guide-on-large-language-models>. [Online]. Available: <https://www.datacamp.com/es/blog/what-is-an-llm-a-guide-on-large-language-models>
- [3] —, “Fine-tuning large language models: A step-by-step guide,” 2024, [Online]. Available: <https://www.datacamp.com/es/tutorial/fine-tuning-large-language-models>. [Online]. Available: <https://www.datacamp.com/es/tutorial/fine-tuning-large-language-models>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, L. Wu, K. Mohta, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *arXiv*, 2020, [Online]. Available: <https://arxiv.org/pdf/2005.11401>. [Online]. Available: <https://arxiv.org/pdf/2005.11401>
- [5] Snorkel AI, “Which is better: Retrieval augmentation (rag) or fine-tuning? both!” 2023, [Online]. Available: <https://snorkel.ai/blog/which-is-better-retrieval-augmentation-rag-or-fine-tuning-both/>. [Online]. Available: <https://snorkel.ai/blog/which-is-better-retrieval-augmentation-rag-or-fine-tuning-both/>
- [6] L. Lavazza and otros, “Icsea 2016: The eleventh international conference on software engineering advances,” in *Proceedings of the ICSEA 2016 Conference*, 2016, accessed on 16 Dec. 2024. [Online]. Available: https://www.researchgate.net/profile/Luigi-Lavazza/publication/307576316_ICSEA_2016_The_Eleventh_International_Conference_on_Software_Engineering_Advances/links/57c9a36a08ae3ac722af8728/ICSEA-2016-The-Eleventh-International-Conference-on-Software-Engineering-Advances.pdf#page=150

- [7] T. Transformation. (2024) Meta’s role in the evolving digital world: Opportunities and challenges. [Online]. Available: <https://tech-transformation.com/marketing-hub/metas-role-in-the-evolving-digital-world-opportunities-and-challenges/>
- [8] N. Osman, “Amazon’s digital transformation,” Institute for Digital Transformation, oct 2023, [Online]. Available: <https://www.institutefordigitaltransformation.org/amazons-digital-transformation/>.
- [9] O. Labs, “Spotify: A great agile example - scrum done right,” OpenView Partners, 2014, [Online]. Available: <https://openviewpartners.com/blog/spotify-great-agile-example-scrum-done-right/>. [Online]. Available: <https://openviewpartners.com/blog/spotify-great-agile-example-scrum-done-right/>
- [10] A. Ahimbisibwe, R. Y. Cavana, and U. Daellenbach, “A contingency fit model of critical success factors for software development projects: A comparison of agile and traditional plan-based methodologies,” *Journal of Enterprise Information Management*, vol. 28, no. 1, pp. 7–33, 2015, [Online]. Available: <https://doi.org/10.1108/JEIM-08-2013-0060>.
- [11] Atlassian, “Product management vs. product development for software teams,” 2024, [Online]. Available: <https://www.atlassian.com/agile/product-management/product-development-software>. [Online]. Available: <https://www.atlassian.com/agile/product-management/product-development-software>
- [12] WRAL TechWire, “Pendo study: With 80 % of features not used, software execs re-evaluating success metrics,” 2020, [Online]. Available: <https://wraltechwire.com/2020/01/28/pendo-study-with-80-of-features-not-used-software-execs-re-evaluating-success-metrics/>. [Online]. Available: <https://wraltechwire.com/2020/01/28/pendo-study-with-80-of-features-not-used-software-execs-re-evaluating-success-metrics/>
- [13] Maturity Model Guy, “The standish group’s 2023 chaos report: Why maturity models matter for project success,” 2023, [Online]. Available: <https://maturitymodelguy.com/the-standish-groups-2023-chaos-report-why-maturity-models-matter-for-project-success/>. [Online]. Available: <https://maturitymodelguy.com/the-standish-groups-2023-chaos-report-why-maturity-models-matter-for-project-success/>
- [14] D. Das, S. Banerjee, M. W. Mahmood, S. Singh, and H. Esmaeilzadeh, “A comprehensive survey on machine learning for computer architecture: 2023 edition,” 2024, [Online]. Available: <https://arxiv.org/abs/2406.00515>. [Online]. Available: <https://arxiv.org/abs/2406.00515>

- [15] —, “Requirements are all you need: From requirements to code with llms,” 2024, [Online]. Available: <https://ar5iv.labs.arxiv.org/html/2406.10101>. [Online]. Available: <https://ar5iv.labs.arxiv.org/html/2406.10101>
- [16] E. M. Bender and A. Koller, “Natural language processing and computational linguistics,” *Computational Linguistics*, vol. 47, no. 4, pp. 707–728, 2021, [Online]. Available: <https://direct.mit.edu/coli/article/47/4/707/107177/Natural-Language-Processing-and-Computational>. [Online]. Available: <https://direct.mit.edu/coli/article/47/4/707/107177/Natural-Language-Processing-and-Computational>
- [17] Y. Goldberg, *Neural Network Methods for Natural Language Processing*. Morgan & Claypool Publishers, 2017, [Online]. Available: https://books.google.cl/books?hl=en&lr=&id=y_lcDwAAQBAJ. [Online]. Available: https://books.google.cl/books?hl=en&lr=&id=y_lcDwAAQBAJ&oi=fnd&pg=PR5&dq=nlp+and+deep+learning&ots=ajsr94SUf7&sig=RAqAgwP4XtF_PJlgAyg0hC-9-A&redir_esc=y#v=onepage&q=nlp%20and%20deep%20learning&f=false
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv*, 2017, [Online]. Available: <https://arxiv.org/abs/1706.03762>. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [19] S. Kublik and S. Saboo, *GPT-3: Building Innovative NLP Products Using Large Language Models*. O’REILLY, 2022, [Online]. Available: <https://books.google.cl/books?hl=en&lr=&id=D2h6EAAAQBAJ&oi=fnd&pg=PT7&dq=GPT+3>. [Online]. Available: <https://books.google.cl/books?hl=en&lr=&id=D2h6EAAAQBAJ&oi=fnd&pg=PT7&dq=GPT+3>
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv*, 2018, [Online]. Available: <https://arxiv.org/abs/1810.04805>. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [21] K. Grover, K. Kaur, K. Tiwari, Rupali, and P. Kumar, “Deep learning based question generation using t5 transformer,” in *Proceedings of International Conference on Frontiers in Computing and Systems*. Springer, 2021, pp. 239–250, [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-16-0401-0_18. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-16-0401-0_18
- [22] V. Zouhar, C. Meister, J. Gastaldi, L. Du, T. Vieira, M. Sachan, and R. Cotterell, “A formal perspective on byte-pair encoding,” in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto, Canada:

Association for Computational Linguistics, 2023, pp. 598–614. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3545945.3569830>

- [23] P. Sharma, “Open ai codex: An inevitable future,” 2023, [Online]. Available: https://www.researchgate.net/publication/368866176_Open_AI_Codex_An_Inevitable_Future. [Online]. Available: https://www.researchgate.net/publication/368866176_Open_AI_Codex_An_Inevitable_Future
- [24] M. R. J, K. VM, and H. W. Y. Gupta, “Fine tuning llm for enterprise: Practical guidelines and recommendations,” *arXiv*, 2024, [Online]. Available: <https://www.datacamp.com/es/tutorial/fine-tuning-large-language-models>. [Online]. Available: <https://arxiv.org/abs/2404.10779>
- [25] Y. K. Lal, T. Gupta, and S. Kumar, “The chronicles of rag: The retriever, the chunk and the generator,” *arXiv*, 2024, [Online]. Available: <https://arxiv.org/pdf/2401.07883>. [Online]. Available: <https://arxiv.org/pdf/2401.07883>
- [26] G. Yenduri, R. M, C. S. G, S. Y, G. Srivastava, P. K. R. Maddikunta, D. R. G, R. H. Jhaveri, P. B, W. Wang, A. V. Vasilakos, and T. R. Gadekallu, “Gpt (generative pre-trained transformer) – a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions,” *arXiv*, 2023, [Online]. Available: <https://arxiv.org/pdf/2305.10435>. [Online]. Available: <https://arxiv.org/pdf/2305.10435>
- [27] U. o. E. School of Informatics, “Introduction to model driven development (mdd),” 2006, [Online]. Available: https://www.inf.ed.ac.uk/teaching/courses/seoc/2006_2007/resources/Mod_MDD1.pdf. [Online]. Available: https://www.inf.ed.ac.uk/teaching/courses/seoc/2006_2007/resources/Mod_MDD1.pdf
- [28] C. C. Rasha Ahmad Husein a, Hala Aburajouh a, “Large language models for code completion: A systematic literature review,” *ScienceDirect*, 2024, [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0920548924000862>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0920548924000862>
- [29] H. Su, S. Jiang, Y. Lai, H. Wu, B. Shi, C. Liu, Q. Liu, and T. Yu, “Arks: Active retrieval in knowledge soup for code generation,” *arXiv preprint arXiv:2406.00515*, 2024. [Online]. Available: <https://arxiv.org/pdf/2402.12317v1>
- [30] L. Zhong and Z. Wang, “Can chatgpt replace stackoverflow? a study on robustness and reliability of large language model code generation,” *arXiv preprint arXiv:2310.03533*, 2024. [Online]. Available: <https://arxiv.org/pdf/2308.10335>

- [31] T. Ahmed, P. Devanbu, C. Treude, and M. Pradel, “Can llms replace manual annotation of software engineering artifacts?” *arXiv preprint arXiv:2408.05534*, 2024, [Online]. Available: <https://arxiv.org/abs/2408.05534>. [Online]. Available: <https://arxiv.org/abs/2408.05534>
- [32] A. B. M. Moniruzzaman and S. Hossain, “Comparative study on agile software development methodologies,” *International Journal of Computer Applications*, July 2013, [Online]. Available: https://www.researchgate.net/publication/249011841_Comparative_Study_on_Agile_software_development_methodologies. [Online]. Available: https://www.researchgate.net/publication/249011841_Comparative_Study_on_Agile_software_development_methodologies